

COMP 3069

Computer Graphics

Lab 7: Cameras

Autumn 2024

0

Contents

- ◆ Use a provided Camera data structure and library function `lookAt()`
- ◆ Use GLFW to check for key presses
- ◆ Implement camera code
- ◆ Apply the Cross-Product operator
- ◆ Calculate points on the unit sphere
- ◆ Demonstrate understanding, implementation, and use of a virtual camera

1

1

1

Set-up of Lab7a

- ◆ Using the instruction in Lab 1, create Lab7a project
- ◆ Download Lab7a.cpp and copy the code to the Lab7a.cpp in the new project
- ◆ Copy the camera.h and ModelViewerCamera.h and add them to the Header Files
- ◆ Download mvp.vert and col.frag, copy them, and add them to the Source Files

Code for Lab7a Default Display

```

20 float vertices[] =
21 {
22     //pos           //col
23     -0.5f, -0.5f, -0.5f, 1.f, 0.0f, 0.0f,
24     0.5f, -0.5f, -0.5f, 1.f, 0.0f, 0.0f,
25     0.5f, 0.5f, -0.5f, 1.f, 0.0f, 0.0f,
26     0.5f, 0.5f, -0.5f, 1.f, 0.0f, 0.0f,
27     -0.5f, 0.5f, -0.5f, 1.f, 0.0f, 0.0f,
28     -0.5f, -0.5f, -0.5f, 1.f, 0.0f, 0.0f,
29
30     -0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
31     0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
32     0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
33     0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
34     -0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
35     -0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
36
37     -0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
38     -0.5f, 0.5f, -0.5f, 0.0f, 0.0f, 1.0f,
39     -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f,
40     -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f,
41     -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
42     -0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
43
44     0.5f, 0.5f, 0.5f, 1.f, 1.0f, 0.0f,
45     0.5f, 0.5f, -0.5f, 1.f, 1.0f, 0.0f,
46     0.5f, -0.5f, -0.5f, 1.f, 1.0f, 0.0f,
47     0.5f, -0.5f, 0.5f, 1.f, 1.0f, 0.0f,
48     0.5f, 0.5f, 0.5f, 1.f, 1.0f, 0.0f,
49     0.5f, 0.5f, -0.5f, 1.f, 1.0f, 0.0f,
50
51     -0.5f, -0.5f, -0.5f, 1.f, 0.0f, 1.0f,
52     0.5f, -0.5f, -0.5f, 1.f, 0.0f, 1.0f,
53     0.5f, 0.5f, -0.5f, 1.f, 0.0f, 1.0f,
54     0.5f, 0.5f, 0.5f, 1.f, 0.0f, 1.0f,
55     -0.5f, 0.5f, 0.5f, 1.f, 0.0f, 1.0f,
56     -0.5f, -0.5f, 0.5f, 1.f, 0.0f, 1.0f,
57
58     -0.5f, 0.5f, -0.5f, 0.0f, 1.f, 1.0f,
59     0.5f, 0.5f, -0.5f, 0.0f, 1.f, 1.0f,
60     0.5f, 0.5f, 0.5f, 0.0f, 1.f, 1.0f,
61     0.5f, 0.5f, 0.5f, 0.0f, 1.f, 1.0f,
62     -0.5f, 0.5f, 0.5f, 0.0f, 1.f, 1.0f,
63     -0.5f, 0.5f, -0.5f, 0.0f, 1.f, 1.0f,
64 };

```

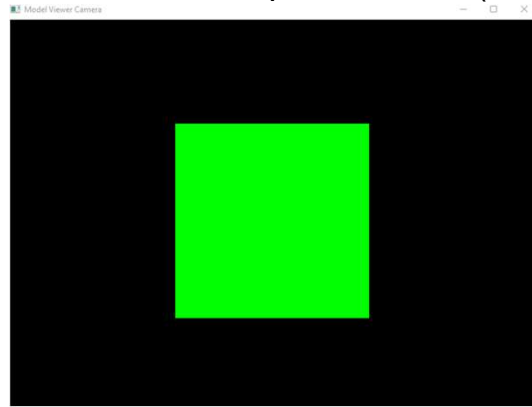
```

318 while (!glfwWindowShouldClose(window))
319 {
320     processKeyboard(window);
321
322     glClearColor(0.f, 0.f, 0.f, 1.f);
323     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
324     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
325
326     glm::mat4 model = glm::mat4(1.f);
327     model = glm::translate(model, cube_pos);
328     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "model"), 1, GL_FALSE, glm::value_ptr(model));
329
330     glm::mat4 view = glm::mat4(1.f);
331     view = glm::translate(view, glm::vec3(8.f, 8.f, 3.f));
332     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
333
334     glm::mat4 projection = glm::mat4(1.f);
335     projection = glm::perspective(glm::radians(45.f), (float)800 / (float)600, 1.f, 10.f);
336     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"), 1, GL_FALSE, glm::value_ptr(projection));
337
338     glBindVertexArray(VAO);
339     glDrawArrays(GL_TRIANGLES, 0, 36);
340
341     glBindVertexArray(0);
342
343     glfwSwapBuffers(window);
344     glfwPollEvents();
345 }

```

Default Display of Lab7a

- ◆ A cube at the origin $(0, 0, 0)$ with multi-coloured faces. The green face of the cube is pointing at the camera, which is positioned at $(0, 0, 3)$.



Implement a Camera

- ◆ A Position in 3D space (3D vector)
- ◆ A Front Direction, (3D vector)
- ◆ A Right Direction, (3D vector)
- ◆ An Up Direction, (3D vector)
- ◆ A Pitch and Yaw value, (float)

camera.h

```
4 struct SCamera
5 {
6     enum Camera_Movement
7     {
8         FORWARD,
9         BACKWARD,
10        LEFT,
11        RIGHT
12    };
13
14    glm::vec3 Position;
15    glm::vec3 Front;
16    glm::vec3 Up;
17    glm::vec3 Right;
18
19    glm::vec3 WorldUp;
20
21    float Yaw;
22    float Pitch;
23
24    const float MovementSpeed = 2.5f;
25    float MouseSensitivity = 1.f;
26
27 };
28
29
30 void InitCamera(SCamera &in)
31 {
32     in.Front = glm::vec3(0.0f, 0.0f, -1.0f);
33     in.Position = glm::vec3(0.0f, 0.0f, 0.0f);
34     in.Up = glm::vec3(0.0f, 1.0f, 0.0f);
35     in.WorldUp = in.Up;
36     in.Right = glm::normalize(glm::cross(in.Front, in.WorldUp));
37
38     in.Yaw = -90.f;
39     in.Pitch = 0.f;
40 }
```

lookAt() etc.

- ◆ A camera object of type SCamera

```
65 SCamera Camera;
66
67
89 InitCamera(Camera);
90 MoveAndOrientCamera(Camera, cube_pos, cam_dist, 0.f, 0.f);
```

- ◆ `lookAt()` constructs a **view matrix** from a camera position, target, and camera up
`mat4 lookAt (vec3 position, vec3 target, vec3 up)`
- ◆ `position` is the position of the camera in world space, `target` is the position of the target of the camera, and `up` is a normalized vector which defines the camera up direction to orient the camera

Model-Viewer Camera

- ◆ In `ModelViewerCamera.h`, there is the function `MoveAndOrientCamera()` to move the camera when user presses a key

```
void MoveAndOrientCamera(Camera cam, vec3 target, float distance, float xoffset, float yoffset)
```
- ◆ `cam` is the camera object, `target` is the position in world space that should be the focus of the camera, `distance` is the distance from the camera to the focused target, `xoffset` and `yoffset` are the amount to move the camera around the imaginary sphere
- ◆ `processKeyboard()` is to detect the user pressing a key and call `MoveAndOrientCamera()` function.

Code Using Model-Viewer Camera

- ◆ In `processKeyboard()`, add some code to detect if the user pressed the RIGHT arrow key and calls the `MoveAndOrientCamera()` function with `x_offset+1`

```
74 void processKeyboard(GLFWwindow* window)
75 {
76     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
77         glfwSetWindowShouldClose(window, true);
78
79     bool cam_changed = false;
80     float x = 0.f, y = 0.f;
81
82     if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
83     {
84         x = 1.f;
85         y = 0.f;
86         cam_changed = true;
87     }
88
89     if (cam_changed)
90     {
91         MoveAndOrientCamera(Camera, cube_pos, cam_dist, x, y);
92     }
93 }
```

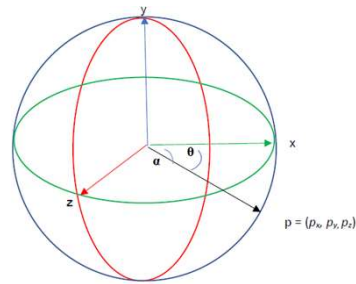
Implementation of Model-Viewer

- ◆ The Model-Viewer camera is implemented using a unit sphere.

$$p_x = \cos \theta * \cos \alpha$$

$$p_y = \sin \alpha$$

$$p_z = \sin \theta * \cos \alpha$$



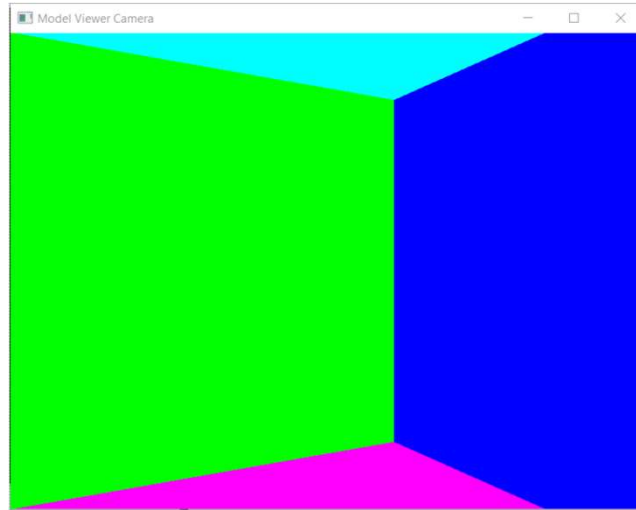
ModelViewerCamera.h

```

7  float cam_dist = 2.f;
8
9  void MoveAndOrientCamera(SCamera& in, glm::vec3 target, float distance, float xoffset, float yoffset)
10 {
11     in.Yaw += xoffset;
12     in.Pitch += yoffset;
13     /*
14     in.Yaw += xoffset * in.MovementSpeed;
15     in.Pitch += yoffset * in.MovementSpeed;
16     */
17     if (in.Pitch > 89.0f) in.Pitch = 89.0f;
18     if (in.Pitch < -89.0f) in.Pitch = -89.0f;
19     /*
20
21     in.Position.x = cos(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch)) * cam_dist;
22     in.Position.y = sin(glm::radians(in.Pitch)) * cam_dist;
23     in.Position.z = sin(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch)) * cam_dist;
24
25     in.Front = glm::normalize(target - in.Position);
26     in.Right = glm::normalize(glm::cross(in.Front, in.WorldUp));
27     in.Up = glm::normalize(glm::cross(in.Right, in.Front))
28 }
29

```

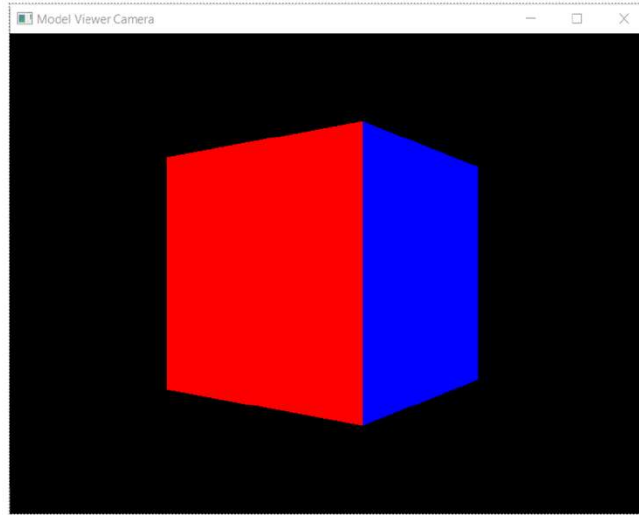
Lab7a Intermediate Display



Camera MovementSpeed

```
1  #pragma once
2
3  #include <stdio.h>
4  #include <glm/glm.hpp>
5  #include "camera.h"
6
7  float cam_dist = 2.f;
8
9  void MoveAndOrientCamera(SCamera& in, glm::vec3 target, float distance, float xoffset, float yoffset)
10 {
11     //in.Yaw -= xoffset;
12     //in.Pitch -= yoffset;
13
14     in.Yaw -= xoffset * in.MovementSpeed;
15     in.Pitch -= yoffset * in.MovementSpeed;
16
17     if (in.Pitch > 89.0f) in.Pitch = 89.0f;
18     if (in.Pitch < -89.0f) in.Pitch = -89.0f;
19
20     in.Position.x = cos(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch)) * cam_dist;
21     in.Position.y = sin(glm::radians(in.Pitch)) * cam_dist;
22     in.Position.z = sin(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch)) * cam_dist;
23
24     in.Front = glm::normalize(target - in.Position);
25     in.Right = glm::normalize(glm::cross(in.Front, in.WorldUp));
26     in.Up = glm::normalize(glm::cross(in.Right, in.Front));
27
28 }
```

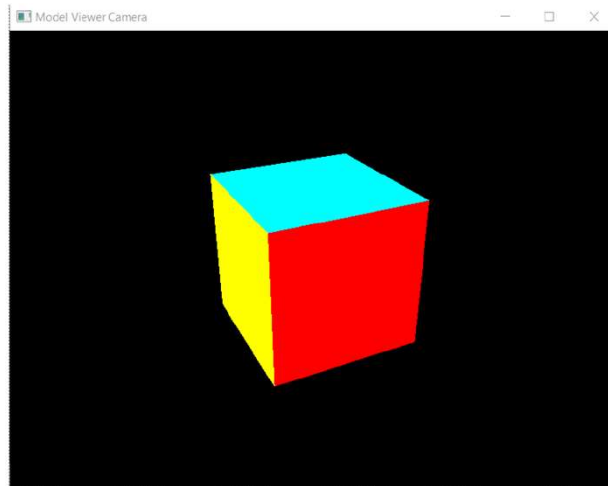
Interactive Displays



Complete processKeyboard()

```
76 void processKeyboard(GLFWwindow* window)
77 {
78     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
79         glfwSetWindowShouldClose(window, true);
80
81     bool cam_changed = false;
82     float x = 0.f, y = 0.f;
83
84
85     if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
86     {
87         x = 0.f;
88         y = -1.f;
89         cam_changed = true;
90     }
91     if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)
92     {
93         x = 0.f;
94         y = 1.f;
95         cam_changed = true;
96     }
97     if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
98     {
99         x = -1.f;
100        y = 0.f;
101        cam_changed = true;
102    }
103    if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
104    {
105        x = 1.f;
106        y = 0.f;
107        cam_changed = true;
108    }
109    if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS)
110    {
111        cam_dist -= 0.1;
112        cam_changed = true;
113    }
114    if (glfwGetKey(window, GLFW_KEY_F) == GLFW_PRESS)
115    {
116        cam_dist += 0.1;
117        cam_changed = true;
118    }
119 }
```


Final Displays



Set-up of Lab7b

- ◆ Using the instruction in Lab 1, create Lab7b project
- ◆ Download `Lab7b.cpp` and copy the code to the `Lab7b.cpp` in the new project
- ◆ Copy the `camera.h` and `FlyThroughCamera.h` and add them to the Header Files
- ◆ Download `mvp.vert` and `col.frag`, copy them, and add them to the Source Files

Default Display



Default FlyThroughCamera.h

```
1  #pragma once
2
3  #include <glm/glm/glm.hpp>
4  #include "camera.h"
5
6  void MoveCamera(SCamera& in, SCamera::Camera_Movement direction)
7  {
8  }
9
10 void OrientCamera(SCamera& in, float xoffset, float yoffset)
11 {
12 }
13
```

Code for Lab7b Default Display

```

74 void processKeyboard(GLFWwindow* window)
75 {
76     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)    glfwSetWindowShouldClose(window, true);
77
78     if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)          MoveCamera(Camera, SCamera::FORWARD);
79     if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)          MoveCamera(Camera, SCamera::BACKWARD);
80     if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)          MoveCamera(Camera, SCamera::LEFT);
81     if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)          MoveCamera(Camera, SCamera::RIGHT);
82 }
83
84 void processMouse(GLFWwindow* window, double x, double y)
85 {
86     if (firstMouse)
87     {
88         prevMouseX = x;
89         prevMouseY = y;
90         firstMouse = false;
91     }
92
93     float dx = x - prevMouseX;
94     float dy = y - prevMouseY;
95
96     prevMouseX = x;
97     prevMouseY = y;
98
99     OrientCamera(Camera, dx, dy);
100 }
101
102 int main(int argc, char** argv)
103 {
104     GLFWwindow* window = CreateWindow(800, 600, "Fly Through Camera");
105     glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
106     glfwSetCursorPosCallback(window, processMouse);
107 }

```

Code in Default Display

```

155 glm::mat4 view = glm::mat4(1.f);
156 view = glm::lookAt(Camera.Position, Camera.Position + Camera.Front, Camera.Up);
157 glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
158
159

```

```

6 void MoveCamera(SCamera& in, SCamera::Camera_Movement direction)
7 {
8 }
9
10 void OrientCamera(SCamera& in, float xoffset, float yoffset)
11 {
12 }

```

Tasks

```
6 void MoveCamera(SCamera& in, SCamera::Camera_Movement direction)
7 {
8     if (direction == SCamera::FORWARD)    in.Position += (in.Front * in.MovementSpeed);
9     if (direction == SCamera::BACKWARD)    in.Position -= (in.Front * in.MovementSpeed);
10    if (direction == SCamera::LEFT)        in.Position -= (in.Right * in.MovementSpeed);
11    if (direction == SCamera::RIGHT)       in.Position += (in.Right * in.MovementSpeed);
12 }
13
14 void OrientCamera(SCamera& in, float xoffset, float yoffset)
15 {
16     in.Yaw += xoffset * in.MouseSensitivity;
17     in.Pitch += yoffset * in.MouseSensitivity;
18
19     if (in.Pitch > 89.0f)    in.Pitch = 89.0f;
20     if (in.Pitch < -89.0f)  in.Pitch = -89.0f;
21
22     glm::vec3 front;
23     front.x = cos(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch));
24     front.y = sin(glm::radians(in.Pitch));
25     front.z = sin(glm::radians(in.Yaw)) * cos(glm::radians(in.Pitch));
26     in.Front = glm::normalize(front);
27
28     in.Right = glm::normalize(glm::cross(in.Front, in.WorldUp));
29
30     in.Up = glm::normalize(glm::cross(in.Right, in.Front));
31 }
```



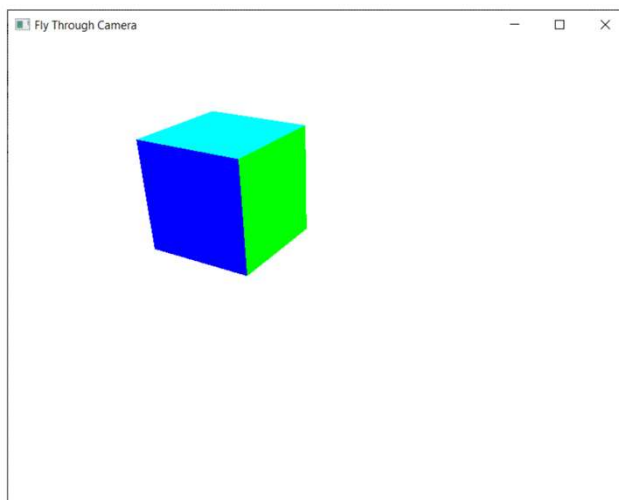
The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

22

22

Outcome



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

23

23

12

References

- ◆ *COMP3069 Lab7 notes*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 4*